



Green Tea GC

what changed, and why it's interesting

hi, I'm Mohammed Nafees

writing Go since 2021

CLIs, k8s operators, Desktop GUIs, security tooling

fell down this rabbit hole reading the 1.25 release notes

a small disclaimer

I am not an expert.

I'm a curious engineer who went down a rabbit hole.

please push back if I get something wrong.

the trail

Aug 2025 — Go 1.25

"experimental: GOEXPERIMENT=greenteagc"

Feb 2026 — Go 1.26

default. every new build is using it.

so: what is it actually doing?

01

a quick GC refresher

(skip ahead in your head if this is old news)

garbage collection

finds memory you no longer use,
reclaims it.

Go's GC algorithm

tri-color concurrent mark-sweep

since Go 1.5 · August 2015

MARK

find what's still alive

SWEEP

reclaim everything else

the punchline of part one

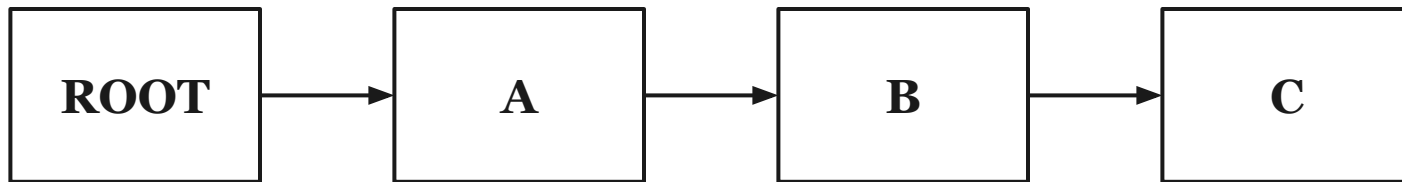
~90%

of GC time is spent marking.

only ~10% sweeping.

if you want a faster GC, you make marking faster.

how marking works



start from roots → follow pointers → mark everything reached

the catch

the order has nothing to do with where things live in memory.

address 0x1000 → 0x800000 → 0x40 (random jumps)

02

the hardware problem

(the part I didn't see coming)

the memory wall

CPUs got fast.

memory didn't keep up.

one trip to RAM $\approx$ hundreds of CPU cycles wasted.

CPUs cope by caching

read one byte →

load a whole cache line of nearby bytes

(betting you'll need them soon)

when the bet pays off — fast.

when it fails — stall.

the killer stat

35%

of GC scan-loop CPU cycles

spent stalled. waiting on memory. doing nothing.

the reframe

the GC isn't slow because
the algorithm is bad.

it's slow because the CPU is sitting idle.

you cannot tune your way out of cache misses.

03

enter green tea

(the actual idea)

***“work with pages,
not objects.”***

— Michael Knyszek, Go runtime team

astonishingly simple. and yet —

a page, in this context

8 KiB

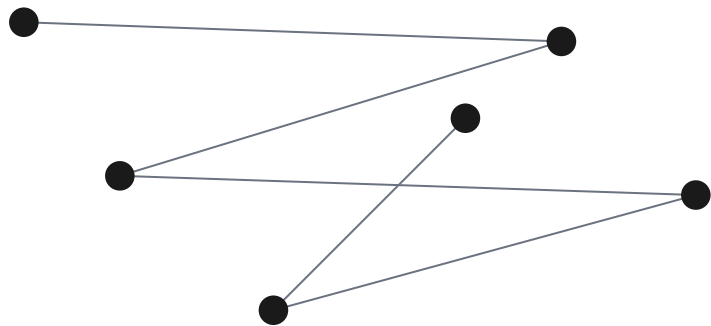
contiguous memory. holds many small objects.

scan all the live ones in one pass.

old vs new

OLD

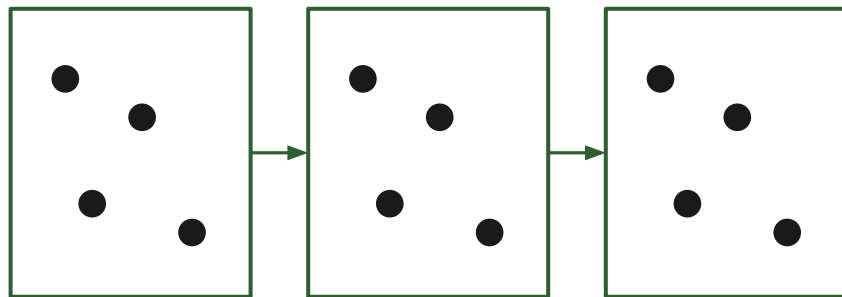
object $\rightarrow$ object $\rightarrow$ object



many cache misses

NEW

page $\rightarrow$ page $\rightarrow$ page



one miss per page

four things to know

1. still tri-color mark-sweep.

pause behavior, GOGC, write barriers — unchanged.

2. small objects only.

large objects scan as before. the two coexist.

3. pages in the work queue.

with goroutine-scheduler-style work stealing.

4. breadth-first across pages.

finish a page before context-switching to a new one.

why this wins

- **cache locality**

subsequent objects on the same page are already cached.

- **the prefetcher can help**

sequential access lets the CPU pull data ahead of time.

- **SIMD on newer x86**

1.26 + Zen 4 / Ice Lake — scan object metadata in parallel.

04

does it actually work?

(receipts.)

the numbers (Go team's benchmark suite)

10–40%

less time in GC.

modal improvement: ~10%

running in production at Google since before 1.25 shipped.

a concrete win: Tile38

~35%

GC overhead reduction. plus better throughput and latency.

why?

a high-fanout tree of small objects = the ideal case.

the honesty slide

DoltHub tried it on their database.
saw essentially no difference.

p50, p99, throughput — all flat.

not every heap shape lets the bet pay off.

more honesty: do the math

10% less GC $\neq$ 10% faster app.

if GC is 10% of your CPU:

$10\% \times 10\% = 1\%$ total CPU saved.

(still real money across a fleet of thousands.)

05

what this means for you

(the practical part.)

Go 1.26

default. just upgrade.

no code changes. no flags. no annotations.

opt out (for now):

```
GOEXPERIMENT=nogreenteagc go build .
```

opt-out goes away in Go 1.27 (August 2026).

how to measure it

simplest:

```
GODEBUG=gctrace=1 ./your-binary
```

more depth:

```
runtime.ReadMemStats    ·    pprof    ·    your APM's GC view
```

GOGC tuning is unchanged. tune like before.

the whole talk in four lines

- modern hardware is memory-bound.
- old GC chased pointers, blew the cache.
- green tea scans pages — fewer stalls.
- 10–40% less GC time, default in 1.26.

thanks

questions?

*(I genuinely don't know why it's called Green Tea.
guesses welcome.)*

sources

go.dev/blog/greenteagc

the official blog post by Michael Knyszek

github.com/golang/go/issues/73581

the design doc and tracking issue

go.dev/doc/go1.26

Go 1.26 release notes

doltHub.com/blog/2025-09-26-greentea-gc-with-dolt

we tried it and saw nothing — honest counter-example

[Michael Knyszek — GopherCon 2025 \(YouTube\)](#)

the original talk this is all based on